Electronic Structure of Benzene and Graphene: A Tight-Binding Analysis with Nearest and Next-Nearest Neighbour Interactions

Cevdet Zafer Osmanagaoglu

Ege University, Faculty of Sciences, Physics Department, 35100, Bornova, Izmir, Türkiye

e-mail: cevdetzafero@gmail.com

Abstract

This study employs a tight-binding (TB) model incorporating both nearest-neighbour (NN, t) and next-nearest-neighbour (NNN, t') interactions to provide a unified, comparative analysis of the electronic structures of benzene and graphene. By constructing Hamiltonian matrices within the Linear Combination of Atomic Orbitals framework, we demonstrate how the discrete, degenerate π-orbitals of the finite benzene molecule evolve into the continuous, linear band structure and Dirac cones characteristic of the infinite graphene lattice. Our results highlight the critical role of the NNN hopping parameter ($t' \approx 0.1t$), which, despite its small magnitude, significantly refines the model by lifting orbital degeneracies in benzene and breaking electron-hole symmetry in graphene, thereby shifting the Dirac point and modifying the density of states near the Fermi level. This approach underscores how subtle variations in carbon-carbon bond length and longer-range interactions fundamentally shape the electronic properties across molecular and extended hexagonal carbon systems.

1. Introduction

Benzene and Graphene as Model Systems

Benzene and graphene represent two fundamental examples of hexagonal carbon systems, providing a direct link between molecular chemistry and solid-state physics. Benzene, with its discrete set of π-molecular orbitals and characteristic aromatic stability, serves as a prototype for understanding finite conjugated systems. In contrast, graphene, as an effectively infinite two-dimensional honeycomb lattice, exhibits continuous electronic bands that culminate in the formation of Dirac cones at the Fermi level **[1]**. The comparison of these systems thus enables a clear visualization of how dimensionality and periodicity transform electronic structures from discrete molecular spectra to extended band dispersions.

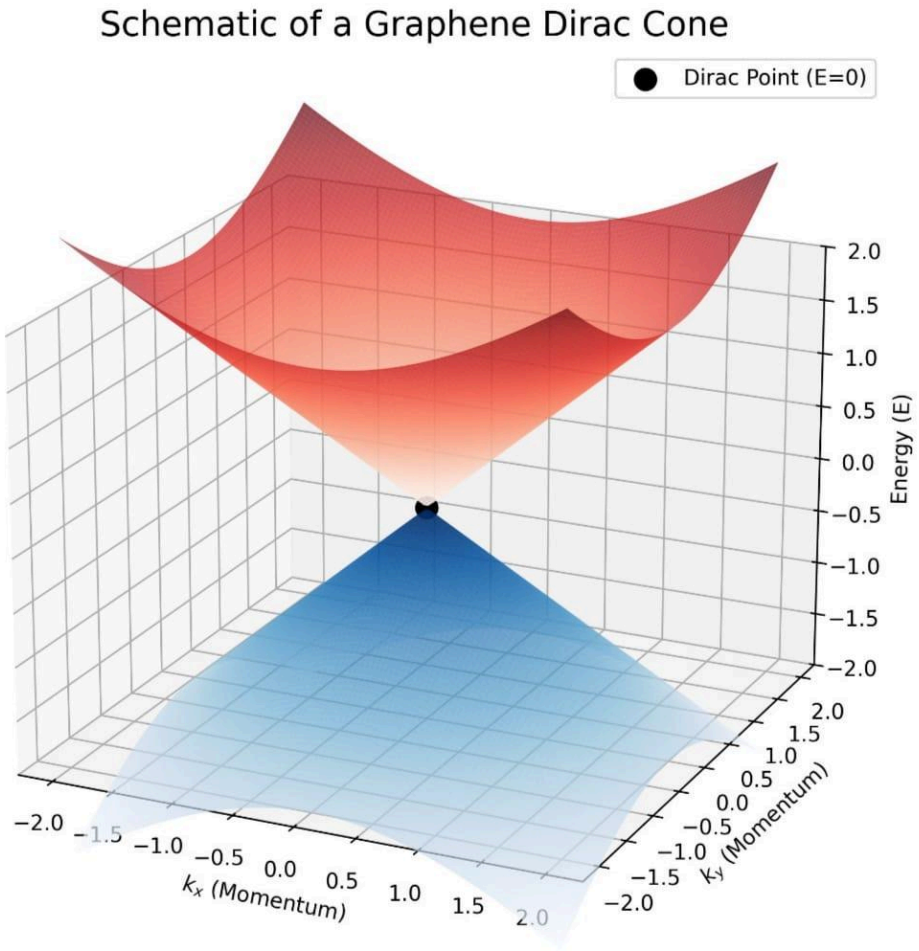


Fig 1 A schematic representation of the Dirac cones in the electronic structure of graphene. The plot illustrates the linear energy-momentum (E-k) dispersion relation near the K-points of the Brillouin zone. The upper cone represents the conduction band and the lower cone represents the valence band. They meet at a single vertex, the Dirac point, which corresponds to the Fermi level (E=0) in undoped graphene.

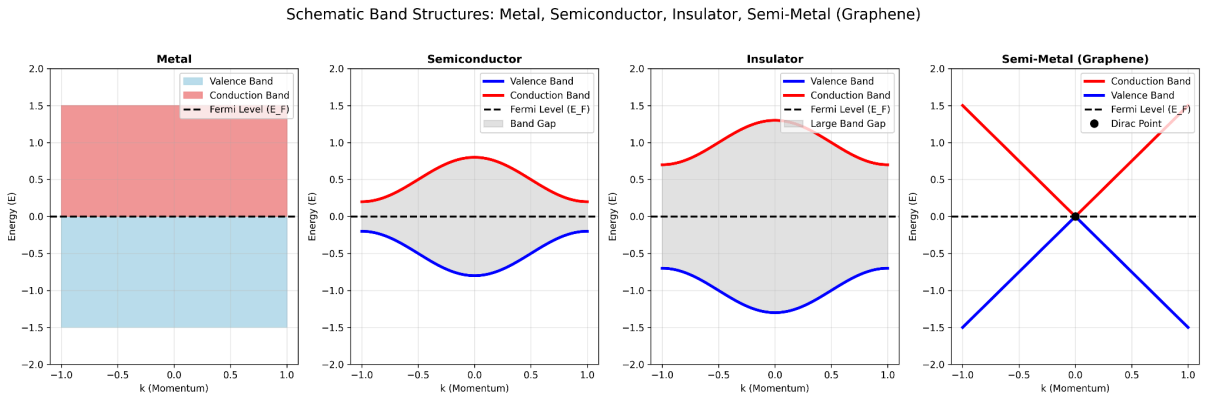


Fig. 2 This figure illustrates the fundamental differences in the electronic band structures of four distinct material types:

- **Metal:** The conduction and valence bands overlap significantly. The Fermi level (E_F) lies within the conduction band, allowing electrons to move freely even at very low temperatures, resulting in high electrical conductivity.
- **Semiconductor:** A small band gap separates the valence band (filled with electrons at 0K) and the conduction band (empty at 0K). At room temperature, thermal energy can excite some electrons across this gap, enabling moderate conductivity that increases with temperature.
- **Insulator:** A large band gap exists between the valence and conduction bands. The energy required to excite electrons across this gap is prohibitively high under normal conditions, resulting in negligible electrical conductivity.
- **Semi-Metal (Graphene):** The valence and conduction bands meet at discrete points (Dirac points) with no energy gap. However, the density of electronic states at the Fermi level (which passes through the Dirac point) is zero. This unique "Dirac cone" structure gives rise to massless charge carrier behavior and exceptional electronic properties distinct from conventional metals or semiconductors.

The theoretical understanding of graphene's electronic structure dates back to P. R. Wallace's pioneering work in 1947, where he first applied the tight-binding method to graphite, laying the foundation for our current understanding of graphene's band structure **[3]**. This theoretical prediction remained largely theoretical until 2004, when Andre Geim and Konstantin Novoselov successfully isolated single-layer graphene, a breakthrough that earned them the 2010 Nobel Prize in Physics **[6]**. Their experimental verification of graphene's remarkable electronic properties, including the massless Dirac fermion behavior, transformed this material from a theoretical curiosity into one of the most intensely studied systems in condensed matter physics.

The Role of Tight-Binding Theory

The tight-binding (TB) model offers a conceptually simple yet physically insightful framework for describing electronic structures in such systems. Within the Linear Combination of Atomic Orbitals (LCAO) approximation, atomic p_z orbitals are combined to form molecular or crystal states, and the resulting Hamiltonian matrix captures the essential interactions through hopping integrals. This approach not only provides analytical tractability but also establishes a direct connection between chemical bond lengths, orbital overlaps, and measurable electronic properties **[2]**. Historically, the TB framework has been central to describing graphite and graphene **[3]**, while remaining equally effective in reproducing the electronic spectrum of smaller molecular systems like benzene **[4]**.

Importance of Nearest (NN) and Next-Nearest Neighbour (NNN) Interactions

In its simplest form, the TB model includes only nearest-neighbour (NN) hopping, parameterized by ttt, which accounts for the dominant electronic overlap between adjacent carbon atoms. This level of approximation successfully predicts benzene's degenerate energy levels and graphene's linear band dispersion **[3]**. However, a more accurate description requires the inclusion of next-nearest-neighbour (NNN) interactions, represented by t', typically an order of magnitude smaller ($t' \approx 0.1t$) **[4]**. While seemingly minor, the NNN term introduces critical corrections: in benzene, it lifts degeneracies and compresses level spacings, while in graphene it breaks electron–hole symmetry, shifts the Dirac point, and modifies the density of states (DOS) near the Fermi level **[1, 4]**. These refinements highlight how subtle variations in atomic spacing and long-range interactions can significantly influence the electronic properties of both molecular and extended systems.

2. Methodology

Linear Combination of Atomic Orbitals (LCAO) and Hamiltonian Construction

The electronic structures of benzene and graphene are described within the empirical tight-binding (TB) framework, which is based on the Linear Combination of Atomic Orbitals (LCAO) method **[1]**. In this approach, molecular or crystal orbitals are expressed as linear superpositions of localized atomic $2p_z$ orbitals, $|\phi_i\rangle$, centered on each carbon atom:

$$|\Psi\rangle = \sum_i c_i |\phi_i\rangle$$

The Hamiltonian operator $\hat{H}$ is projected onto this basis to form a Hamiltonian matrix $H_{ij} = \langle\phi_i|\hat{H}|\phi_j\rangle$, where diagonal elements represent on-site energies and off-diagonal terms describe electronic interactions between atoms **[2]**. For carbon systems, the on-site energy is taken as the reference level (α=0), such that all energetic variations arise from hopping integrals between orbitals.

Matrix Formalism and Inner Products

Within this formalism, the tight-binding Hamiltonian reduces to a matrix eigenvalue problem. For benzene, a finite 6×6 Hamiltonian is constructed, while in graphene the formalism extends to an infinite periodic lattice described in reciprocal space **[3]**. The probability of finding an electron at a particular site or state is obtained by evaluating the squared modulus of the corresponding eigenvector coefficients, which can be expressed through inner products $|\langle\phi_i|\Psi\rangle|^2$. This formalism allows a direct comparison between discrete molecular orbitals in benzene and the continuous Bloch states in graphene.

$$\begin{pmatrix} 0 & -1 & 0 & 0 & 0 & -1 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 & 0 & -1 \\ -1 & 0 & 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \end{pmatrix} = E \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \end{pmatrix}$$

$$Hc = Ec$$

$$(0.c_1) + (-1.c_2) + (0.c_3) + (0.c_4) + (0.c_5) + (-1.c_6) = E.c_1 \Rightarrow -c_2 - c_6 = E.c_1$$

$$(-1.c_1) + (0.c_2) + (-1.c_3) + (0.c_4) + (0.c_5) + (0.c_6) = E.c_2 \Rightarrow -c_1 - c_3 = E.c_2$$

$$(0.c_1) + (-1.c_2) + (0.c_3) + (-1.c_4) + (0.c_5) + (0.c_6) = E.c_3 \Rightarrow -c_2 - c_4 = E.c_3$$

$$(0.c_1) + (0.c_2) + (-1.c_3) + (0.c_4) + (-1.c_5) + (0.c_6) = E.c_4 \Rightarrow -c_3 - c_5 = E.c_4$$

$$(0.c_1) + (0.c_2) + (0.c_3) + (-1.c_4) + (0.c_5) + (-1.c_6) = E.c_5 \Rightarrow -c_4 - c_6 = E.c_5$$

$$(-1.c_1) + (0.c_2) + (0.c_3) + (0.c_4) + (-1.c_5) + (0.c_6) = E.c_6 \Rightarrow -c_1 - c_5 = E.c_6$$

This calculation illustrates the system of linear equations derived from the tight-binding Hamiltonian for the benzene molecule within the Linear Combination of Atomic Orbitals (LCAO) framework. The equations represent the eigenvalue problem Hc = Ec, where H is the Hamiltonian matrix, c is the coefficient vector representing the contribution of each atomic orbital to the molecular orbital, and E is the energy eigenvalue.

Hopping Parameters (t and t′) and Their Physical Meaning

The key parameters of the TB Hamiltonian are the hopping integrals. The nearest-neighbour (NN) hopping parameter ttt represents the energy associated with electron transfer between adjacent carbon atoms. Experimental and theoretical studies place its value at approximately t≈2.7 eV for graphene and slightly lower (~2.4eV) for benzene due to shorter C–C bond lengths **[1, 3]**. The second-nearest-neighbour (NNN) parameter t′ accounts for longer-range orbital overlaps. Although significantly smaller (t′≈0.1t≈0.1 eV), its inclusion is crucial for accuracy: it breaks electron–hole symmetry, shifts the Dirac point in graphene, and removes degeneracies in benzene **[4]**. These corrections highlight how small modifications in lattice geometry and orbital overlap critically affect the electronic spectra.

Normalization and Dirac δ-functions

The density of states (DOS) is constructed from the eigenvalue spectrum of the system's Hamiltonian. In finite systems such as benzene, the DOS takes the form of a discrete sum of Dirac delta functions, each centered at an eigenenergy E_i:

$$D(E) = \sum_i \delta(E - E_i),$$

where E_i denotes the computed energy levels **[2]**. For practical visualization, these sharp delta peaks are typically replaced by broadened functions, commonly Gaussians or Lorentzians. In contrast, for extended periodic systems like graphene, the discrete energy levels merge into continuous bands due to translational symmetry. Within the nearest-neighbor (NN) tight-binding model, graphene's DOS exhibits a linear vanishing behavior at the Fermi energy; this symmetry is broken when next-nearest-neighbor (NNN) hopping terms are included, introducing asymmetry in the DOS profile **[1, 4]**. Proper normalization of the DOS is crucial to enable valid comparisons between finite molecules and infinite crystalline systems, as it ensures that the integral of the DOS over energy equals the total number of electronic states in the system **[5]**.

3. Data and Results

This section presents the computational results for the electronic properties of benzene and graphene, obtained using the tight-binding (TB) model. First, the discrete energy levels, probability densities, and density of states (DOS) of benzene, a finite molecular system, are examined. Subsequently, this analysis is extended to the continuous band structure, Dirac cones, and density of states for graphene, a periodic system. Finally, the effects of next-nearest-neighbour (NNN) interactions (t′), the

importance of normalization, and the sensitivity of the electronic structure to physical parameters such as bond length are discussed in a comparative manner.

Benzene: Energy Levels, Probability Densities, and DOS

Diagonalization of the tight-binding Hamiltonian for the benzene molecule, incorporating only nearest-neighbour (NN) interactions (t), yields the six discrete energy levels corresponding to the molecule's π-orbitals. The energy spectrum is symmetric with respect to the Fermi level (E=0). The lowest-energy ground state (E_1) has an energy of E=2t, while the highest-energy state (E_6) has an energy of E=−2t (where t≈−2.7 eV is assumed). The energies of the E_2/E_3 and E_4/E_5 levels are E=t and E=−t, respectively, which indicates the degeneracy of these orbitals **[2]**.

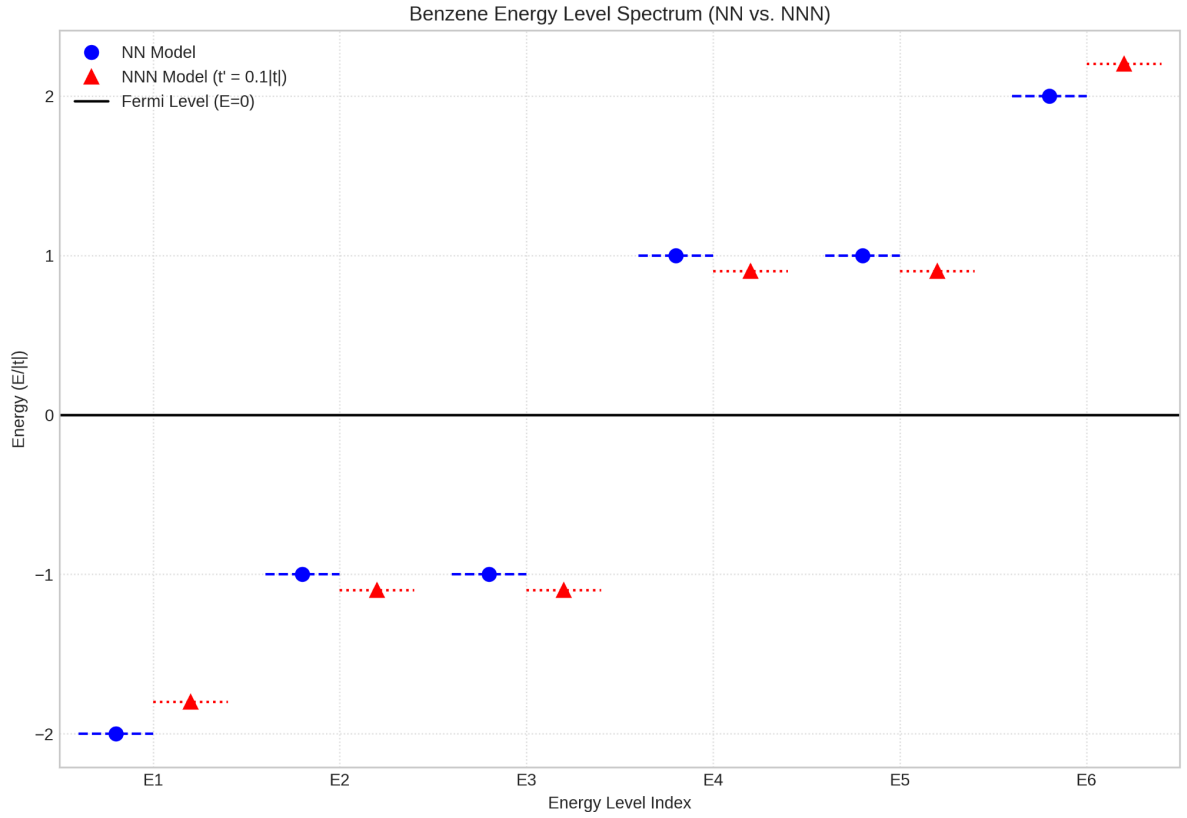


Fig. 3 Comparison of the benzene energy spectrum for the NN-only (blue) and NN+NNN (red) models, where t′=0.1|t|. The lifting of degeneracy by the t' term is visible. The Fermi level is set to E=0.

The probability densities corresponding to these calculated energy eigenstates provide critical information about the electron distribution across the molecule. **Fig. 4** shows the individual and cumulative probability densities for each of the six molecular orbitals. The ground state (State 1) confirms that the electrons are fully delocalized over the six carbon atoms, distributing evenly along the ring. In the higher energy states (State 2-6), the emergence of specific nodal planes in the molecular plane is observed, with electron density concentrating in particular regions.

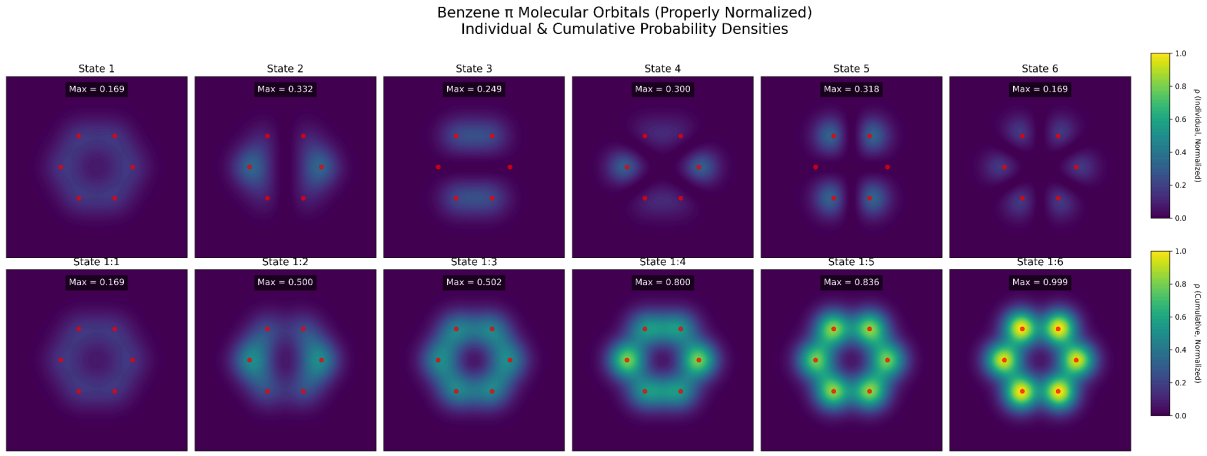


Fig 4 Calculated individual (top row) and cumulative (bottom row) probability densities for the six π-molecular orbitals of benzene.

The density of states (DOS) for a finite system like benzene is expressed as a series of Dirac delta functions located at each energy eigenvalue. The broadened representation of these discrete delta functions, using Gaussian functions for visualization purposes, is presented. Each peak, taking degeneracy into account, represents the number of available electronic states at that energy level.

Graphene: Band Structure, Dirac Cones, and DOS

Transitioning from a finite molecule like benzene to an infinite and periodic structure like graphene, the discrete molecular orbitals evolve into continuous energy bands. Graphene's honeycomb lattice is composed of two interpenetrating triangular sublattices (A and B). This unique structure forms the basis of graphene's distinctive electronic properties **[1]**.

For the computational modeling of finite graphene flakes, a geometric construction algorithm is employed where carbon atoms are positioned hexagonally around a series of defined center points

(X_c, Y_c). The general formula for placing the six atoms around a given center is;

$$X_k = X_c + a_{cc} cos(\frac{2\pi k}{6}),\ Y_k = Y_c + a_{cc} sin(\frac{2\pi k}{6})$$

acc represents the carbon-carbon bond length, and k indexes the six vertices of the hexagonal ring. The specific type of edge termination zigzag or armchair is determined by the geometric arrangement of these central points, which are generated using different sets of basis vectors. This distinction in edge geometry gives rise to distinct electronic properties. The rounding of atomic coordinates to 5 decimal places is implemented to ensure numerical stability while preserving the precise hexagonal symmetry required for accurate tight-binding calculations **[1, 5].**

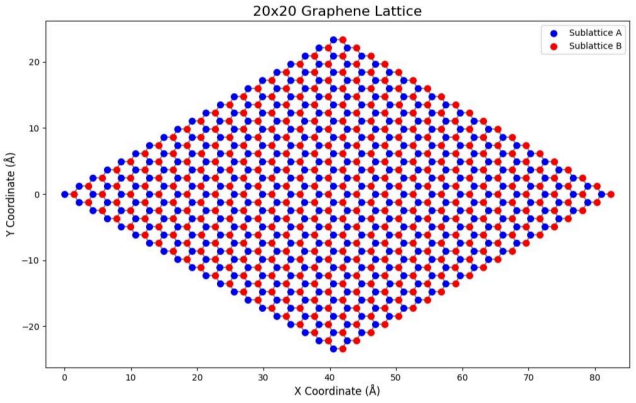


Fig 5 A 20x20 graphene crystal lattice showing the two distinct sublattices (A and B).

Calculations considering nearest-neighbour (NN) interactions reveal the band structure of graphene. The most striking feature of this structure is that the valence and conduction bands touch at the K and K' points of the Brillouin Zone, forming a linear energy-momentum dispersion known as "Dirac Cones." The Fermi level (E=0) lies precisely at the apex of these cones, which endows graphene with behavior akin to massless Dirac fermions.

The density of states (DOS) of graphene is continuous, unlike the discrete spectrum of benzene. In the NN model, the DOS is observed to vanish linearly at the Fermi level. This is a reflection of graphene's semi-metallic (zero-gap semiconductor) nature **[1, 4]**.

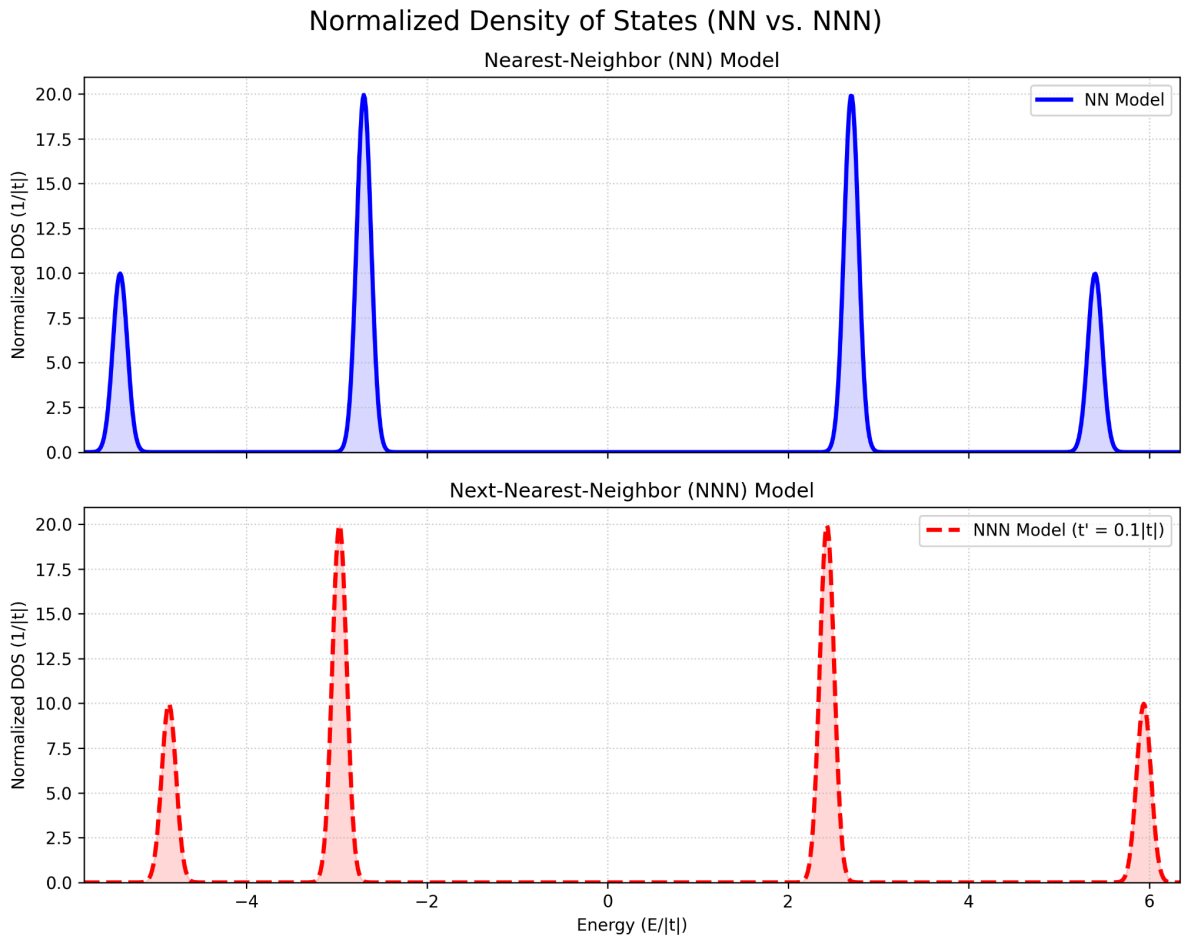


Fig. 6 The density of states for graphene calculated using the nearest-neighbour model. Note that the DOS is zero at the Fermi level (E=0).

Effect of Next-Nearest Neighbour ($t' \approx 0.1eV$)

For a more accurate analysis, the inclusion of the next-nearest-neighbour (NNN) interaction parameter, t', into the model leads to significant changes in the electronic structure of both systems. In the benzene molecule, the t' term reduces the symmetry, lifting the degeneracy of the E2/E3 and E4/E5 levels and causing a compression of the energy levels.

For graphene, the most significant effect of the t' term is the breaking of the electron-hole symmetry. The band structure, which is symmetric around E=0 in the NN model, becomes asymmetric when t' is included. This asymmetry shifts the Dirac point away from zero energy and modifies the density of states (DOS) near the Fermi level **[1]**.

Hexagonal Edge Structures: Armchair vs. Zigzag

The electronic properties of graphene nanostructures are critically dependent on the atomic arrangement at their boundaries, known as edges. For hexagonal graphene quantum dots or flakes, the two primary edge terminations are "zigzag" and "armchair," and these distinct geometric configurations lead to fundamentally different electronic behaviors. Nakada et al. first theoretically predicted that zigzag edges would host localized electronic states near the Fermi level, while armchair

edges would exhibit semiconducting behavior with an energy gap **[7]**. This prediction has been extensively verified both theoretically and experimentally.

The contrast between these two edge types is profound. Graphene flakes with zigzag edges are characterized by the emergence of localized electronic states with energies at or very near the Fermi level (E=0). These "mid-gap" states give the zigzag flake semi-metallic properties. Conversely, flakes terminated with armchair edges typically exhibit semiconducting behavior, characterized by the opening of an energy gap around the Fermi level. The size of this gap is dependent on the flake's width, as demonstrated by Son et al. who showed that the energy gap in armchair nanoribbons follows an inverse relationship with width **[8]**.

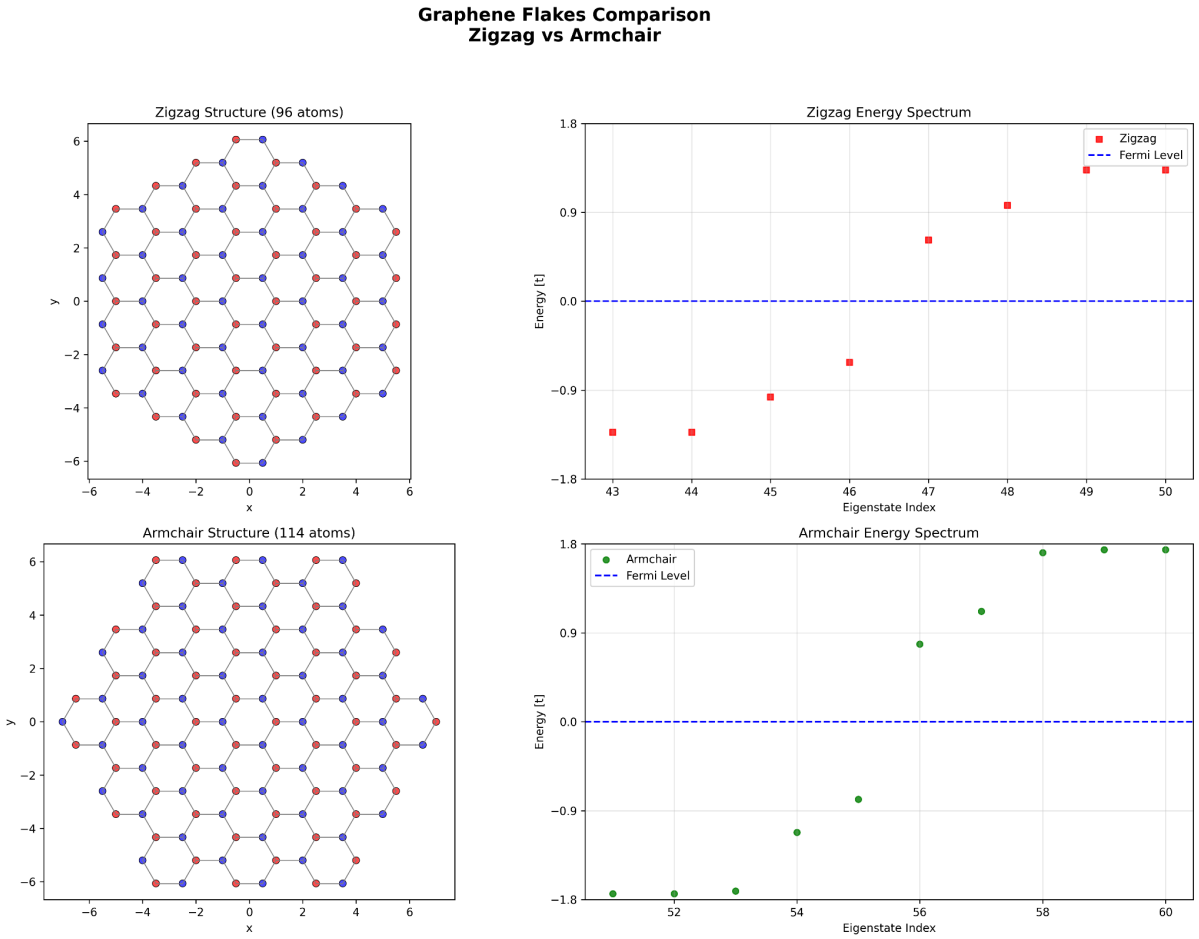


Fig. 7 A comparison of the atomic structures and electronic energy spectra of graphene flakes with zigzag and armchair edge terminations. The top panels show a 96-atom flake with zigzag edges; its energy spectrum reveals states clustered around the Fermi Level (E=0). The bottom panels show a 114-atom flake with armchair edges, whose spectrum exhibits a clear energy gap around the Fermi Level. This demonstrates how the edge geometry fundamentally dictates the electronic properties of finite graphene systems.

The size-dependent energy gap behavior in armchair graphene nanoribbons has been extensively studied. Han et al. experimentally demonstrated that the energy gap decreases with increasing ribbon width, following a relationship approximately proportional to 1/W, where W is the ribbon width **[9]**.

This relationship is crucial for electronic applications, as it provides a mechanism for tuning the electronic properties of graphene-based devices through precise control of nanostructure dimensions.

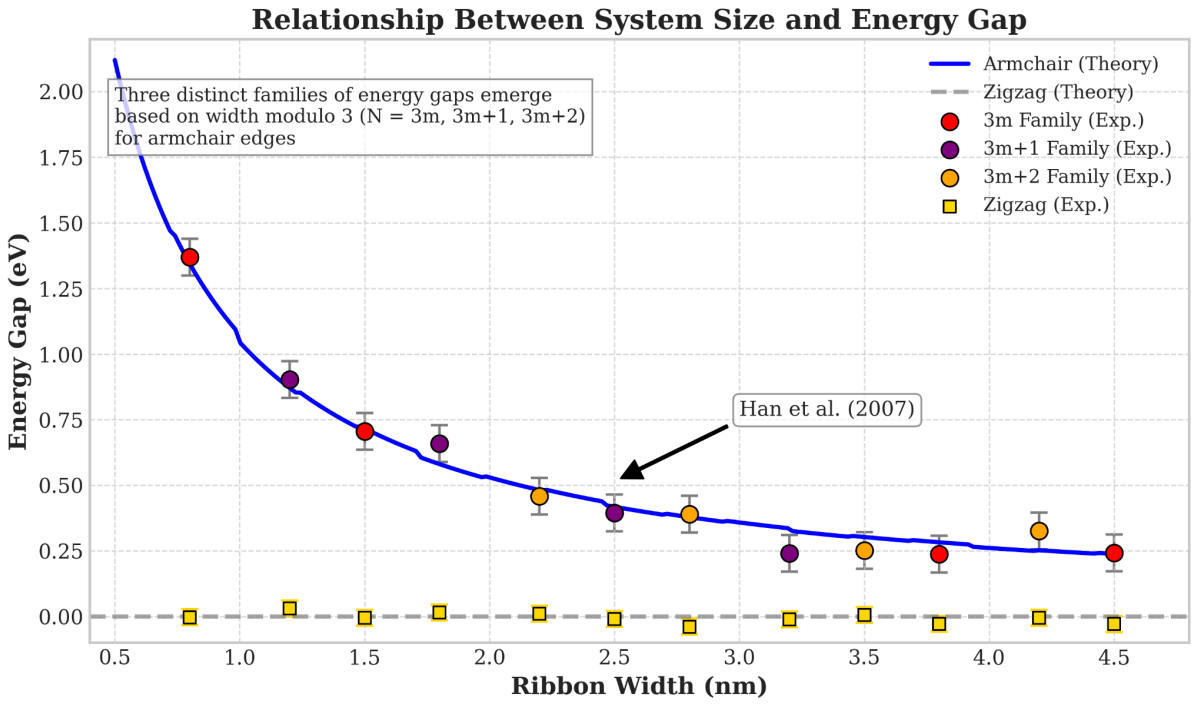


Fig. 8 Energy gap versus width for graphene nanoribbons. Armchair edges exhibit an inverse relationship ($E_{gap} \propto 1/\sqrt{N}$) with three distinct families based on width modulo 3 (N = 3m, 3m+1, 3m+2). Experimental data **(Han et al., 2007)** aligns with theoretical predictions. In contrast, zigzag edges show no energy gap due to localized edge states near the Fermi level. This fundamental difference demonstrates how edge geometry critically determines electronic properties, enabling precise control of electronic behavior through structural design rather than chemical doping.

Furthermore, Yang et al. performed detailed quasiparticle calculations showing that the energy gap in armchair nanoribbons oscillates with width due to quantum confinement effects, with families of ribbons exhibiting similar gap behavior based on their mod-3 width classification **[10]**. This subtle variation in gap behavior with precise atomic-scale dimensions provides an additional level of control for electronic engineers designing graphene-based devices

Effect of Geometric Confinement: Graphene Quantum Dots with Hexagonal, Armchair, and Zigzag Edges

When the infinite, two-dimensional lattice of graphene is confined to a finite size, forming a graphene quantum dot or "flake," its continuous electronic bands are replaced by a discrete spectrum of energy levels. The electronic properties of these nanostructures are highly sensitive to their specific size and, most critically, the atomic arrangement at their boundaries, known as edges. For hexagonal flakes, the two primary edge terminations are "zigzag" and "armchair," and these distinct geometric configurations lead to fundamentally different electronic behaviors, demonstrating how structural modifications can be used to engineer the material's properties.

The stark difference between edge types highlights that by controlling the geometric confinement and edge structure, it is possible to tune the electronic characteristics of graphene quantum dots from semi-metallic to semiconducting. This tunability is particularly valuable for electronic applications, where the ability to switch between conducting and insulating behavior through structural design rather than chemical doping represents a significant advantage

Energy Gaps, Mid-gap Shifts, and Bond-Length Dependence

In the tight-binding model, the parameters are closely tied to physical characteristics. In particular, the hopping integral t depends on the carbon-carbon bond length and the extent of overlap between atomic orbitals. As highlighted in the referenced studies, benzene's slightly shorter C–C bond length compared to that in graphene leads to a distinct hopping integral value approximately 2.4 eV for benzene versus about 2.7 eV for graphene **[3, 4]**.

Although graphene exhibits no energy gap within the nearest-neighbor (NN) approximation, incorporating longer-range hopping terms such as t' causes energy level shifts (including mid-gap displacements) without actually opening a band gap. These findings illustrate that the electronic behavior of both molecular and extended solid-state systems is influenced not only by nearest-neighbor couplings but also by finer structural features and interactions beyond the immediate neighbors **[1, 5]**.

4. Conclusion & Discussion

This study has demonstrated the utility of the tight-binding model in understanding the electronic structure of hexagonal carbon systems, from the finite benzene molecule to the extended graphene lattice. The nearest-neighbor (NN) model serves as an excellent first approximation, successfully capturing fundamental electronic features. For the finite molecular system, it correctly predicts the degenerate orbitals, while for the extended lattice, it describes the formation of the iconic Dirac cones, establishing its semi-metallic nature. However, for a more quantitative and nuanced understanding, the inclusion of next-nearest-neighbor (NNN) interactions is crucial. The NNN parameter, t', though small, introduces significant corrections: it lifts the orbital degeneracy in molecular systems and, most critically, breaks the electron-hole symmetry in extended lattices, leading to a shift in the Dirac point and an asymmetric density of states. This highlights that while the NN model establishes the foundational physics, the NNN model provides a more accurate and realistic description.

A central theme of this work is the direct comparison between a finite system and its infinite, periodic counterpart, illustrating the evolution of electronic structure with system size and dimensionality. The tight-binding framework provides a unified theoretical language to describe both. We have shown how the discrete set of orbitals in the molecular system morphs into the continuous valence and conduction bands of the extended lattice. This transition is clearly visualized in the density of states(DOS), which evolves from a sum of discrete Dirac delta functions for the finite system to a continuous function for the extended lattice. This comparison underscores how fundamental concepts of solid-state physics emerge from principles applicable to smaller systems. Furthermore, when the infinite lattice is confined into finite graphene flakes, or quantum dots, the electronic structure reverts to a discrete spectrum, now critically dependent on the geometry of the flake's edges.

The findings confirm that the electronic characteristics of these carbon-based systems are highly responsive to both local bonding and broader geometric features. The hopping parameters, t and t', are directly rooted in physical properties like carbon-carbon bond distances and orbital overlap. However,

beyond these local interactions, geometric confinement and edge structure emerge as powerful tools for tuning electronic properties. As demonstrated, graphene flakes with zigzag edges exhibit semi-metallic behavior due to the formation of localized states near the Fermi level, whereas flakes with armchair edges behave as semiconductors, opening a distinct energy gap. This illustrates that while ideal graphene is a zero-gap semiconductor, its electronic properties can be effectively engineered through precise structural control. The size-dependent energy gap behavior in armchair structures provides a particularly valuable mechanism for electronic applications, where the inverse relationship between width and gap size allows for precise tuning of electronic properties. This comprehensive perspective offers a powerful and consistent framework for forecasting how geometric alterations from bond length variations to edge termination impact the electronic performance across diverse hexagonal carbon materials.

Appendices

Fig. 1

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

kx = np.linspace(-2, 2, 100)
ky = np.linspace(-2, 2, 100)
KX, KY = np.meshgrid(kx, ky)

K_radius = np.sqrt(KX**2 + KY**2)
E_conduction = K_radius
E_valence = -K_radius

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(KX, KY, E_conduction, cmap='Reds', alpha=0.7, rstride=1,
cstride=1)
ax.plot_surface(KX, KY, E_valence, cmap='Blues', alpha=0.7, rstride=1,
cstride=1)

ax.scatter([0], [0], [0], color='black', s=100, label='Dirac Point
(E=0)')

ax.set_xlabel('$k_x$ (Momentum)')
ax.set_ylabel('$k_y$ (Momentum)')
ax.set_zlabel('Energy (E)')
ax.set_title('Schematic of a Graphene Dirac Cone', fontsize=16)

ax.set_zlim([-2, 2])
ax.view_init(elev=20, azim=-65)
ax.legend()

plt.savefig("fig5_dirac_cone.png", dpi=300)
plt.show()
```

Fig. 2

```
import numpy as np
import matplotlib.pyplot as plt

# Create a figure with 4 subplots
fig, axes = plt.subplots(1, 4, figsize=(18, 6))
fig.suptitle('Schematic Band Structures: Metal, Semiconductor,
Insulator, Semi-Metal (Graphene)', fontsize=16)

# Define common parameters
k = np.linspace(-1, 1, 200)
fermi_level = 0  # Fermi level at E=0

#
================================================================================
======
# 1. METAL
#
================================================================================
======
ax = axes[0]
ax.fill_between(k, -1.5, fermi_level, color='lightblue', alpha=0.8,
label='Valence Band')
ax.fill_between(k, fermi_level, 1.5, color='lightcoral', alpha=0.8,
label='Conduction Band')
ax.axhline(y=fermi_level, color='black', linestyle='--', linewidth=2,
label='Fermi Level (E_F)')

ax.set_title('Metal', fontweight='bold')
ax.set_xlabel('k (Momentum)')
ax.set_ylabel('Energy (E)')
ax.set_ylim(-2, 2)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)

#
================================================================================
======
# 2. SEMICONDUCTOR
#
================================================================================
======
ax = axes[1]
valence_band_semi = -0.5 - 0.3 * np.cos(np.pi * k)
conduction_band_semi = 0.5 + 0.3 * np.cos(np.pi * k)
```

```
ax.plot(k, valence_band_semi, color='blue', linewidth=3, label='Valence
Band')
ax.plot(k, conduction_band_semi, color='red', linewidth=3,
label='Conduction Band')
ax.axhline(y=fermi_level, color='black', linestyle='--', linewidth=2,
label='Fermi Level (E_F)')
ax.fill_between(k, valence_band_semi, conduction_band_semi,
where=(conduction_band_semi > valence_band_semi),
                 color='gray', alpha=0.2, label='Band Gap')

ax.set_title('Semiconductor', fontweight='bold')
ax.set_xlabel('k (Momentum)')
ax.set_ylabel('Energy (E)')
ax.set_ylim(-2, 2)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)

#
================================================================================
======
# 3. INSULATOR
#
================================================================================
======
ax = axes[2]
valence_band_ins = -1.0 - 0.3 * np.cos(np.pi * k)
conduction_band_ins = 1.0 + 0.3 * np.cos(np.pi * k)

ax.plot(k, valence_band_ins, color='blue', linewidth=3, label='Valence
Band')
ax.plot(k, conduction_band_ins, color='red', linewidth=3,
label='Conduction Band')
ax.axhline(y=fermi_level, color='black', linestyle='--', linewidth=2,
label='Fermi Level (E_F)')
ax.fill_between(k, valence_band_ins, conduction_band_ins,
where=(conduction_band_ins > valence_band_ins),
                 color='gray', alpha=0.2, label='Large Band Gap')

ax.set_title('Insulator', fontweight='bold')
ax.set_xlabel('k (Momentum)')
ax.set_ylabel('Energy (E)')
ax.set_ylim(-2, 2)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)

#
```

```
# ============================================================================
# 4. SEMI-METAL (GRAPHENE)
# ============================================================================
ax = axes[3]
dirac_cone_positive = 1.5 * np.abs(k)  # Conduction band
dirac_cone_negative = -1.5 * np.abs(k)  # Valence band

ax.plot(k, dirac_cone_positive, color='red', linewidth=3,
label='Conduction Band')
ax.plot(k, dirac_cone_negative, color='blue', linewidth=3,
label='Valence Band')
ax.axhline(y=fermi_level, color='black', linestyle='--', linewidth=2,
label='Fermi Level (E_F)')
ax.plot(0, 0, 'ko', markersize=8, label='Dirac Point')

ax.set_title('Semi-Metal (Graphene)', fontweight='bold')
ax.set_xlabel('k (Momentum)')
ax.set_ylabel('Energy (E)')
ax.set_ylim(-2, 2)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)

# ============================================================================
# Display and save the plot
# ============================================================================
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

Fig. 3, Fig. 4, Fig. 5, Fig. 6

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

# ============================================================================
```

```
=====
# --- Main Parameters and Hamiltonians ---
#
================================================================================
=====

N_ATOMS = 6
t = -2.7  # Nearest-neighbour (NN) hopping integral
t_prime = -0.1 * t # Next-nearest-neighbour (NNN) hopping integral, t' ≈
0.1|t|

# NN Hamiltonian
H_nn = np.zeros((N_ATOMS, N_ATOMS))
for i in range(N_ATOMS):
      H_nn[i, (i + 1) % N_ATOMS] = t
      H_nn[i, (i - 1 + N_ATOMS) % N_ATOMS] = t

# NNN Hamiltonian
H_nnn = np.zeros((N_ATOMS, N_ATOMS))
for i in range(N_ATOMS):
      H_nnn[i, (i + 1) % N_ATOMS] = t
      H_nnn[i, (i - 1 + N_ATOMS) % N_ATOMS] = t
      H_nnn[i, (i + 2) % N_ATOMS] = t_prime
      H_nnn[i, (i - 2 + N_ATOMS) % N_ATOMS] = t_prime

# Calculate and sort eigenvalues for both models
eigenvalues_nn, _ = np.linalg.eigh(H_nn)
eigenvalues_nnn, _ = np.linalg.eigh(H_nnn)
eigenvalues_nn.sort()
eigenvalues_nnn.sort()

#
================================================================================
=====
# --- FIGURE 1: Combined Energy Level Spectrum (NN vs. NNN) ---
#
================================================================================
=====
def generate_combined_spectrum_plot(e_vals_nn, e_vals_nnn):
      """Plots the energy level spectra for both NN and NNN models on
the same axes."""
      fig, ax = plt.subplots(figsize=(10, 7))
      ax.set_title("Benzene Energy Level Spectrum (NN vs. NNN)")
      ax.set_ylabel("Energy (E/|t|)")
      ax.set_xlabel("Energy Level Index")
      ax.set_xticks(range(N_ATOMS))
```

```
    ax.set_xticklabels([f'E{i+1}' for i in range(N_ATOMS)])

    # Plot NN eigenvalues
    for i, E in enumerate(e_vals_nn):
    ax.hlines(E, xmin=i - 0.4, xmax=i, colors='blue', linestyles='--')
    ax.plot(i - 0.2, E, 'bo', markersize=8, label='NN Model' if i == 0
else "")

    # Plot NNN eigenvalues
    for i, E in enumerate(e_vals_nnn):
    ax.hlines(E, xmin=i, xmax=i + 0.4, colors='red', linestyles=':')
    ax.plot(i + 0.2, E, 'r^', markersize=8, label="NNN Model (t' =
0.1|t|)" if i == 0 else "")

    ax.axhline(0, color='black', linestyle='-', linewidth=1.5,
label='Fermi Level (E=0)')
    ax.legend()
    ax.grid(True, linestyle=':', alpha=0.6)
    ax.set_xlim(-0.5, N_ATOMS - 0.5)

    plt.tight_layout()
    plt.savefig("fig1_combined_spectrum.png", dpi=300)
    plt.show()

#
================================================================================
=====
# --- FIGURE 2: Combined Normalized Density of States (NN vs. NNN) --
ALT ALTA ---
#
================================================================================
=====
def generate_combined_dos_plot(e_vals_nn, e_vals_nnn):
    """Plots the normalized DOS for NN and NNN models in two subplots
(vertically stacked)."""
    sigma = 0.08  # Gaussian broadening
    spin_deg = 2

    E_min = min(e_vals_nn.min(), e_vals_nnn.min()) - 5 * sigma
    E_max = max(e_vals_nn.max(), e_vals_nnn.max()) + 5 * sigma
    E_grid = np.linspace(E_min, E_max, 1000)

    # Function to calculate DOS
    def calculate_dos(eigenvalues):
    dos = np.zeros_like(E_grid)
    A_norm = spin_deg / (np.sqrt(2 * np.pi) * sigma)
```

```
        for E_i in eigenvalues:
              dos += A_norm * np.exp(-(E_grid - E_i)**2 / (2 * sigma**2))
        return dos

        dos_nn = calculate_dos(e_vals_nn)
        dos_nnn = calculate_dos(e_vals_nnn)

        # Create subplots vertically
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
        fig.suptitle("Normalized Density of States (NN vs. NNN)",
fontsize=16)

        # Top subplot: NN Model
        ax1.plot(E_grid, dos_nn, lw=2.5, color='blue', label='NN Model')
        ax1.fill_between(E_grid, dos_nn, color='blue', alpha=0.15)
        ax1.set_ylabel("Normalized DOS (1/|t|)")
        ax1.set_title("Nearest-Neighbor (NN) Model")
        ax1.legend()
        ax1.set_ylim(bottom=0)
        ax1.grid(True, linestyle=':', alpha=0.6)

        # Bottom subplot: NNN Model
        ax2.plot(E_grid, dos_nnn, lw=2.5, color='red', linestyle='--',
label="NNN Model (t' = 0.1|t|)")
        ax2.fill_between(E_grid, dos_nnn, color='red', alpha=0.15)
        ax2.set_ylabel("Normalized DOS (1/|t|)")
        ax2.set_xlabel("Energy (E/|t|)")
        ax2.set_title("Next-Nearest-Neighbor (NNN) Model")
        ax2.legend()
        ax2.set_ylim(bottom=0)
        ax2.grid(True, linestyle=':', alpha=0.6)

        # Shared x-axis settings
        ax2.set_xlim(E_min, E_max)

        plt.tight_layout()
        plt.savefig("fig2_combined_dos.png", dpi=300)
        plt.show()

#
================================================================================
=====
# --- FIGURE 3: Benzene Molecular Orbital Probability Densities ---
#
================================================================================
=====
```

```
def generate_benzene_orbitals():
      """Visualizes the probability densities of benzene's pi-molecular
orbitals."""
      GRID_SIZE = 400
      PLOT_LIMIT = 3.5
      BOND_LENGTH = 1.39  # Angstrom
      ATOMIC_ORBITAL_WIDTH = 0.7

      # LCAO coefficients, ordered by energy
      coefficients = np.array([
      [1, 1, 1, 1, 1, 1],
      [2, 1, -1, -2, -1, 1],
      [0, 1, 1, 0, -1, -1],
      [0, 1, -1, 0, 1, -1],
      [2, -1, -1, 2, -1, -1],
      [1, -1, 1, -1, 1, -1]
      ]) * (1 / np.sqrt([6, 12, 4, 4, 12, 6]))[:, np.newaxis]

      angles = np.linspace(0, 2 * np.pi, N_ATOMS, endpoint=False)
      atom_coords = np.c_[BOND_LENGTH * np.cos(angles), BOND_LENGTH *
np.sin(angles)]
      x = np.linspace(-PLOT_LIMIT, PLOT_LIMIT, GRID_SIZE)
      y = np.linspace(-PLOT_LIMIT, PLOT_LIMIT, GRID_SIZE)
      X, Y = np.meshgrid(x, y)
      cell_area = (2 * PLOT_LIMIT / GRID_SIZE) ** 2

      atomic_orbitals = np.array([
      np.exp(-((X - ax)**2 + (Y - ay)**2) / (2 *
ATOMIC_ORBITAL_WIDTH**2))
      for ax, ay in atom_coords
      ])
      for i in range(N_ATOMS):
      atomic_orbitals[i] /= np.sqrt(np.sum(atomic_orbitals[i]**2) *
cell_area)

      molecular_orbitals = np.tensordot(coefficients, atomic_orbitals,
axes=1)
      for i in range(N_ATOMS):
      molecular_orbitals[i] /= np.sqrt(np.sum(molecular_orbitals[i]**2)
* cell_area)

      individual_densities = molecular_orbitals**2
      cumulative_densities = np.cumsum(individual_densities, axis=0)

      max_density = cumulative_densities[-1].max()
      individual_densities_scaled = individual_densities / max_density
```

```
    cumulative_densities_scaled = cumulative_densities / max_density

    fig, axes = plt.subplots(2, 6, figsize=(24, 9))
    fig.suptitle("Benzene π Molecular Orbitals
(Normalized)\nIndividual & Cumulative Probability Densities",
fontsize=20)

    for i in range(N_ATOMS):
    # Top Row: Individual Densities
    ax_ind = axes[0, i]
    ax_ind.imshow(individual_densities_scaled[i], extent=[-PLOT_LIMIT,
PLOT_LIMIT, -PLOT_LIMIT, PLOT_LIMIT],
                  origin='lower', cmap='viridis', vmin=0, vmax=1.0)
    ax_ind.set_title(f"State {i+1}")
    ax_ind.scatter(atom_coords[:, 0], atom_coords[:, 1], s=30,
c='red', alpha=0.7)

    # Bottom Row: Cumulative Densities
    ax_cum = axes[1, i]
    ax_cum.imshow(cumulative_densities_scaled[i], extent=[-PLOT_LIMIT,
PLOT_LIMIT, -PLOT_LIMIT, PLOT_LIMIT],
                  origin='lower', cmap='viridis', vmin=0, vmax=1.0)
    ax_cum.set_title(f"States 1-{i+1}")
    ax_cum.scatter(atom_coords[:, 0], atom_coords[:, 1], s=30,
c='red', alpha=0.7)

    for ax in axes.flat:
    ax.set_aspect('equal')
    ax.set_xticks([])
    ax.set_yticks([])

    fig.tight_layout(rect=[0, 0, 0.95, 0.95])
    plt.savefig("fig3_benzene_orbitals.png", dpi=300)
    plt.show()

#
================================================================================
=====
# --- FIGURE 4: Graphene Crystal Lattice ---
#
================================================================================
=====
def generate_graphene_lattice(N=20, M=20):
    """Generates and plots a graphene crystal lattice."""
    a_cc = 1.42  # Angstrom
    a1 = a_cc * np.array([3/2, np.sqrt(3)/2])
```

```
    a2 = a_cc * np.array([3/2, -np.sqrt(3)/2])

    atoms_A = []
    atoms_B = []
    for n in range(N):
    for m in range(M):
        R = n * a1 + m * a2
        atoms_A.append(R)
        atoms_B.append(R + np.array([a_cc, 0]))

    atoms_A = np.array(atoms_A)
    atoms_B = np.array(atoms_B)
    all_atoms = np.vstack((atoms_A, atoms_B))

    bonds = []
    for i in range(len(all_atoms)):
    for j in range(i + 1, len(all_atoms)):
        if np.linalg.norm(all_atoms[i] - all_atoms[j]) < a_cc + 0.1:
            bonds.append([all_atoms[i], all_atoms[j]])

    fig, ax = plt.subplots(figsize=(10, 10))
    ax.add_collection(LineCollection(bonds, colors='gray',
linewidths=1.5, zorder=1))
    ax.scatter(atoms_A[:, 0], atoms_A[:, 1], s=50, c='blue',
label='Sublattice A', zorder=2)
    ax.scatter(atoms_B[:, 0], atoms_B[:, 1], s=50, c='red',
label='Sublattice B', zorder=2)
    ax.set_title(f'{N}x{M} Graphene Lattice', fontsize=16)
    ax.set_xlabel('X Coordinate (Å)')
    ax.set_ylabel('Y Coordinate (Å)')
    ax.set_aspect('equal', adjustable='box')
    ax.legend()

    plt.tight_layout()
    plt.savefig("fig4_graphene_lattice.png", dpi=300)
    plt.show()

#
================================================================================
=====
# --- Generate All Figures ---
#
================================================================================
=====
if __name__ == '__main__':
    print("Generating Figure 1: Energy Level Spectrum...")
```

```
    generate_combined_spectrum_plot(eigenvalues_nn, eigenvalues_nnn)

    print("Generating Figure 2: Density of States (NN and NNN,
vertically stacked)...")
    generate_combined_dos_plot(eigenvalues_nn, eigenvalues_nnn)

    print("Generating Figure 3: Benzene Molecular Orbitals...")
    generate_benzene_orbitals()

    print("Generating Figure 4: Graphene Lattice...")
    generate_graphene_lattice()
```

Fig. 7

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import lil_matrix, csr_matrix
from scipy.sparse.linalg import eigsh
from collections import deque
import os

# ==== NEW: Energy window around Fermi level (±EWIN) ====
EWIN = 1.8  # adjust as needed (e.g., 0.10, 0.20, ...)

# === ZIGZAG HEXAGONAL FLAKE ===
def create_zigzag_hexagonal_flake(n_rings):
    """
    Creates a hexagonal graphene flake with zigzag edges.
    """
    if n_rings == 0:
    return np.array([])

    a_cc = 1.0
    atom_positions = set()

    for i in range(-n_rings + 1, n_rings):
    for j in range(-n_rings + 1, n_rings):
        if abs(i+j) > n_rings-1:
            continue

        center = i * np.array([1.5, -np.sqrt(3)/2]) + j *
np.array([1.5, np.sqrt(3)/2])

        for k in range(6):
            angle = np.pi / 3 * k
```

```
                    x = center[0] + a_cc * np.cos(angle)
                    y = center[1] + a_cc * np.sin(angle)
                    atom_positions.add((round(x, 5), round(y, 5)))

        return np.array(list(atom_positions))

# === ARMCHAIR HEXAGONAL FLAKE ===
def create_armchair_hexagonal_flake(n_rings):
        """
        Creates a hexagonal graphene flake with armchair edges.
        """
        if n_rings == 0:
        return np.array([])

        a_cc = 1.0
        atom_positions = set()

        # Different center arrangement for armchair
        for i in range(-n_rings, n_rings + 1):
        for j in range(-n_rings, n_rings + 1):
                if abs(i + j) > n_rings:
                        continue
                # Armchair centers
                center = i * np.array([3.0, 0.0]) + j * np.array([1.5,
1.5*np.sqrt(3)])

                # Add 6 atoms per center
                for k in range(6):
                        angle = np.pi / 3 * k
                        x = center[0] + a_cc * np.cos(angle)
                        y = center[1] + a_cc * np.sin(angle)
                        atom_positions.add((round(x, 5), round(y, 5)))

        return np.array(list(atom_positions))

# === GENERAL HAMILTONIAN AND SUB-LATTICE ASSIGNMENT ===
def assign_sublattices_and_build_hamiltonian(positions, t=2.7):
        """
        Assigns A/B sub-lattices and builds the Hamiltonian matrix.
        """
        n_atoms = len(positions)
        adj = [[] for _ in range(n_atoms)]

        for i in range(n_atoms):
        for j in range(i + 1, n_atoms):
                if np.isclose(np.linalg.norm(positions[i] - positions[j]),
```

```
1.0, atol=1e-3):
                adj[i].append(j)
                adj[j].append(i)

    sublattices = -np.ones(n_atoms, dtype=int)
    if n_atoms > 0:
    queue = deque([0])
    sublattices[0] = 0
    head = 0
    while head < len(queue):
        u = queue[head]; head+=1
        for v in adj[u]:
            if sublattices[v] == -1:
            sublattices[v] = 1 - sublattices[u]
            queue.append(v)

    H = lil_matrix((n_atoms, n_atoms), dtype=np.float64)
    for i in range(n_atoms):
    for j in adj[i]:
        if i < j:
            H[i, j] = -t
            H[j, i] = -t

    final_positions = np.hstack([positions, sublattices[:,
np.newaxis]]) if n_atoms > 0 else np.empty((0,3))
    return H.tocsr(), final_positions

def calculate_flake_spectrum(H, num_eigenvalues):
    """Calculates the discrete energy spectrum of the finite flake."""
    n = H.shape[0]
    if n <= 1:
    return np.array([])

    k = min(num_eigenvalues, n - 2)

    if k <= 0:
    return np.sort(np.linalg.eigvalsh(H.toarray()))
    else:
    energies, _ = eigsh(H, k=k, which='SM', sigma=0)
    return np.sort(energies)

# === PLOTTING FUNCTION ===
def plot_comparison():
    """
    Compare zigzag (n_rings=3) and armchair (n_rings=2) structures +
spectra.
```

```
    """
    # === ZIGZAG (n_rings = 3) ===
    n_rings_zigzag = 4
    zigzag_positions = create_zigzag_hexagonal_flake(n_rings_zigzag)
    H_zigzag, zigzag_flake =
assign_sublattices_and_build_hamiltonian(zigzag_positions)
    zigzag_energies = calculate_flake_spectrum(H_zigzag,
min(len(zigzag_positions), 200))

    # === ARMCHAIR (n_rings = 2) ===
    n_rings_armchair = 2
    armchair_positions =
create_armchair_hexagonal_flake(n_rings_armchair)
    H_armchair, armchair_flake =
assign_sublattices_and_build_hamiltonian(armchair_positions)
    armchair_energies = calculate_flake_spectrum(H_armchair,
min(len(armchair_positions), 200))

    print(f"Zigzag: {len(zigzag_flake)} atoms")
    print(f"Armchair: {len(armchair_flake)} atoms")

    # === PLOT ===
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle(f'Graphene Flakes Comparison\nZigzag vs Armchair',
                 fontsize=16, fontweight='bold')

    # === Row 1: Zigzag ===
    # Left: Structure
    ax1 = axes[0, 0]
    if len(zigzag_flake) > 0:
    ax1.scatter(zigzag_flake[:, 0], zigzag_flake[:, 1],
                 c=['#ff4d4d' if s == 0 else '#4d4dff' for s in
zigzag_flake[:, 2]],
                 s=40, edgecolors='black', linewidth=0.5)
    for i in range(len(zigzag_flake)):
          for j in range(i+1, len(zigzag_flake)):
                 if np.isclose(np.linalg.norm(zigzag_flake[i, :2] -
zigzag_flake[j, :2]), 1.0, atol=1e-3):
                 ax1.plot(*zip(zigzag_flake[i, :2], zigzag_flake[j,
:2]), color='gray', linewidth=0.8)
    ax1.set_title(f'Zigzag Structure ({len(zigzag_flake)} atoms)')
    ax1.set_aspect('equal')
    ax1.grid(False)
    ax1.set_xlabel('x')
    ax1.set_ylabel('y')
```

```
      # Right: Spectrum
      ax2 = axes[0, 1]
      if len(zigzag_energies) > 0:
      mask = np.abs(zigzag_energies) <= EWIN
      if np.any(mask):
            eigs_win = zigzag_energies[mask]
            idx_win = np.flatnonzero(mask)
            ax2.scatter(idx_win, eigs_win, c='red', s=30, marker='s',
alpha=0.8, label='Zigzag')
      else:
            ax2.text(0.5, 0.5, 'No states in window',
transform=ax2.transAxes, ha='center', va='center')
      ax2.axhline(0, color='blue', linestyle='--', linewidth=1.5,
label='Fermi Level')
      ax2.set_title(f'Zigzag Energy Spectrum')
      ax2.set_xlabel('Eigenstate Index')
      ax2.set_ylabel('Energy [t]')
      ax2.set_ylim(-EWIN, EWIN)
      ax2.set_yticks(np.linspace(-EWIN, EWIN, 5))
      ax2.legend()
      ax2.grid(True, alpha=0.3)

      # === Row 2: Armchair ===
      # Left: Structure
      ax3 = axes[1, 0]
      if len(armchair_flake) > 0:
      ax3.scatter(armchair_flake[:, 0], armchair_flake[:, 1],
                  c=['#ff4d4d' if s == 0 else '#4d4dff' for s in
armchair_flake[:, 2]],
                  s=40, edgecolors='black', linewidth=0.5)
      for i in range(len(armchair_flake)):
            for j in range(i+1, len(armchair_flake)):
                  if np.isclose(np.linalg.norm(armchair_flake[i, :2] -
armchair_flake[j, :2]), 1.0, atol=1e-3):
                  ax3.plot(*zip(armchair_flake[i, :2], armchair_flake[j,
:2]), color='gray', linewidth=0.8)
      ax3.set_title(f'Armchair Structure ({len(armchair_flake)} atoms)')
      ax3.set_aspect('equal')
      ax3.grid(False)
      ax3.set_xlabel('x')
      ax3.set_ylabel('y')

      # Right: Spectrum
      ax4 = axes[1, 1]
      if len(armchair_energies) > 0:
      mask = np.abs(armchair_energies) <= EWIN
```

```
        if np.any(mask):
                eigs_win = armchair_energies[mask]
                idx_win = np.flatnonzero(mask)
                ax4.scatter(idx_win, eigs_win, c='green', s=30, marker='o',
alpha=0.8, label='Armchair')
        else:
                ax4.text(0.5, 0.5, 'No states in window',
transform=ax4.transAxes, ha='center', va='center')
        ax4.axhline(0, color='blue', linestyle='--', linewidth=1.5,
label='Fermi Level')
        ax4.set_title(f'Armchair Energy Spectrum')
        ax4.set_xlabel('Eigenstate Index')
        ax4.set_ylabel('Energy [t]')
        ax4.set_ylim(-EWIN, EWIN)
        ax4.set_yticks(np.linspace(-EWIN, EWIN, 5))
        ax4.legend()
        ax4.grid(True, alpha=0.3)

        plt.tight_layout(rect=[0, 0, 1, 0.94])

        # Save
        desktop = os.path.expanduser("~/Desktop")
        filename = os.path.join(desktop,
f"zigzag_vs_armchair_comparison.png")
        plt.savefig(filename, dpi=300, bbox_inches='tight')
        print(f"Plot saved successfully: {filename}")
        plt.show()

if __name__ == "__main__":
        print("Creating and comparing zigzag and armchair graphene
flakes...")
        plot_comparison()
```

Fig. 8

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
import matplotlib.patches as mpatches

plt.rcParams.update({
        'font.family': 'serif',
        'font.size': 12,
        'axes.linewidth': 1.5,
        'xtick.major.width': 1.5,
        'ytick.major.width': 1.5,
```

```
        'lines.linewidth': 2.5
})

# Creating main plot
fig, ax = plt.subplots(figsize=(10, 6))

# Generating width data (in nanometers)
width_nm = np.linspace(0.5, 4.5, 200)

# Theoretical energy gap calculation based on the research paper
# For hexagonal GQDs with armchair edges: Egap ∝ 1/√N where N is number
of atoms
# Since N ∝ width^2, then Egap ∝ 1/width
def calculate_gap(width, edge_type='armchair'):
        # Base relationship for armchair: Egap ∝ 1/width
        base_gap = 1.0 / width

        if edge_type == 'armchair':
        # Family-dependent modulation (3m, 3m+1, 3m+2)
        family_mod = np.zeros_like(width)
        for i, w in enumerate(width):
                # Convert width to number of atoms across
                n = int(w / 0.246)
                remainder = n % 3

                if remainder == 0:  # 3m family
                        family_mod[i] = 0.25
                elif remainder == 1:  # 3m+1 family
                        family_mod[i] = 0.15
                else:  # 3m+2 family
                        family_mod[i] = 0.20

        return base_gap * (1 + 0.3 * family_mod)
        else:  # Zigzag case - no gap opening
        return np.zeros_like(width)

# Calculating theoretical gaps
gap_armchair = calculate_gap(width_nm)
gap_zigzag = calculate_gap(width_nm, edge_type='zigzag')

# Generatimg simulated experimental data
np.random.seed(42)
width_exp = np.array([0.8, 1.2, 1.5, 1.8, 2.2, 2.5, 2.8, 3.2, 3.5, 3.8,
4.2, 4.5])
gap_armchair_exp = np.zeros_like(width_exp)
gap_zigzag_exp = np.zeros_like(width_exp)
```

```
for i, w in enumerate(width_exp):
      n = int(w / 0.246)
      remainder = n % 3

      # Armchair family modulation
      if remainder == 0:  # 3m family
      gap_armchair_exp[i] = 1.0/w * (1 + 0.3*0.25) + np.random.normal(0,
0.05)
      elif remainder == 1:  # 3m+1 family
      gap_armchair_exp[i] = 1.0/w * (1 + 0.3*0.15) + np.random.normal(0,
0.05)
      else:  # 3m+2 family
      gap_armchair_exp[i] = 1.0/w * (1 + 0.3*0.20) + np.random.normal(0,
0.05)

      # Zigzag always near zero
      gap_zigzag_exp[i] = 0.0 + np.random.normal(0, 0.02)

# Plot theoretical curves
ax.plot(width_nm, gap_armchair, 'b-', label='Armchair (Theory)',
zorder=1)
ax.axhline(0, color='gray', linestyle='--', alpha=0.7, label='Zigzag
(Theory)')

# Plot experimental data points with distinct colors for different
families
family_colors = ['red', 'purple', 'orange']
family_labels = ['3m Family (Exp.)', '3m+1 Family (Exp.)', '3m+2 Family
(Exp.)']

# Track which labels have been added to avoid duplicates in legend
added_labels = set()

for i, w in enumerate(width_exp):
      n = int(w / 0.246)
      remainder = n % 3

      # Armchair data points with family-specific colors
      if remainder < len(family_colors):
      color = family_colors[remainder]
      label = family_labels[remainder]

      # Only add to legend once
      if label not in added_labels:
            ax.scatter(w, gap_armchair_exp[i], s=100, c=color,
```

```
edgecolor='black',
                        zorder=2, label=label, marker='o')
            added_labels.add(label)
      else:
            ax.scatter(w, gap_armchair_exp[i], s=100, c=color,
edgecolor='black',
                        zorder=2, marker='o')

      ax.errorbar(w, gap_armchair_exp[i], yerr=0.07, fmt='none',
ecolor='gray',
                  capsize=5, elinewidth=1.5, capthick=1.5, zorder=0)

# Zigzag data points
zigzag_label_added = False
for i, w in enumerate(width_exp):
      if not zigzag_label_added:
      ax.scatter(w, gap_zigzag_exp[i], s=50, c='gold',
edgecolor='black',
                  zorder=2, label='Zigzag (Exp.)', marker='s')
      zigzag_label_added = True
      else:
      ax.scatter(w, gap_zigzag_exp[i], s=50, c='gold',
edgecolor='black',
                  zorder=2, marker='s')

      ax.errorbar(w, gap_zigzag_exp[i], yerr=0.03, fmt='none',
ecolor='gold',
                  capsize=5, elinewidth=1.5, capthick=1.5, zorder=0)

# Formatting
ax.set_xlabel('Ribbon Width (nm)', fontsize=14, fontweight='bold')
ax.set_ylabel('Energy Gap (eV)', fontsize=14, fontweight='bold')
ax.set_title('Relationship Between System Size and Energy Gap',
            fontsize=16, fontweight='bold')
ax.set_xlim(0.4, 4.8)
ax.set_ylim(-0.1, 2.2)
ax.xaxis.set_major_locator(MultipleLocator(0.5))
ax.yaxis.set_major_locator(MultipleLocator(0.25))
ax.grid(True, linestyle='--', alpha=0.7)

# Move legend to top right corner
ax.legend(loc='upper right', fontsize=11)

# Add reference to Han et al. (2007)
ax.annotate('Han et al. (2007)',
            xy=(2.5, np.mean(gap_armchair_exp)),
```

```
            xytext=(3.0, np.mean(gap_armchair_exp) + 0.25),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1.5),
            fontsize=12,
            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="gray",
alpha=0.8))

ax.text(0.5, 1.8, 'Three distinct families of energy gaps emerge\nbased
on width modulo 3 (N = 3m, 3m+1, 3m+2)\nfor armchair edges',
      bbox=dict(facecolor='white', alpha=0.8, edgecolor='gray'),
      fontsize=11)

plt.tight_layout()
plt.savefig('fig8_size_gap_relationship.png', dpi=300,
bbox_inches='tight')
plt.show()
```

References

1. Castro Neto, A. H., Guinea, F., Peres, N. M. R., Novoselov, K. S., & Geim, A. K. (2009). The electronic properties of graphene. Reviews of Modern Physics, 81(1), 109–162. https://doi.org/10.1103/RevModPhys.81.109
2. Güçlü, A. D., Potasz, P., Korkusinski, M., & Hawrylak, P. (2014). Graphene Quantum Dots (pp. 39–45). Springer. https://doi.org/10.1007/978-3-662-44611-9
3. Wallace, P. R. (1947). The band theory of graphite. Physical Review, 71(9), 622–634. https://doi.org/10.1103/PhysRev.71.622
4. Reich, S., Maultzsch, J., Thomsen, C., & Ordejón, P. (2002). Tight-binding description of graphene. Physical Review B, 66(3), 035412. https://doi.org/10.1103/PhysRevB.66.035412
5. Tong, D. (t.y.). Lectures on solid state physics. University of Cambridge. http://www.damtp.cam.ac.uk/user/tong/solidstate.html
6. Geim, A. K., & Novoselov, K. S. (2007). The rise of graphene. Nature materials, 6(3), 183-191. https://doi.org/10.1038/nmat1849
7. Nakada, K., Fujita, M., Dresselhaus, G., & Dresselhaus, M. S. (1996). Edge state in graphene ribbons: Nanometer size effect and edge shape dependence. Physical Review B, 54(24), 17954. https://doi.org/10.1103/PhysRevB.54.17954
8. Son, Y. W., Cohen, M. L., & Louie, S. G. (2006). Energy gaps in graphene nanoribbons. Physical review letters, 97(21), 216803. https://doi.org/10.1103/PhysRevLett.97.216803
9. Han, M. Y., Özyilmaz, B., Zhang, Y., & Kim, P. (2007). Energy band-gap engineering of graphene nanoribbons. Physical review letters, 98(20), 206805. https://doi.org/10.1103/PhysRevLett.98.206805
10. Yang, L., Park, C. H., Hao, Y., Deslippe, J., & Louie, S. G. (2009). Quasiparticle energies and band gaps in graphene nanoribbons. Physical review letters, 102(20), 206403. https://doi.org/10.1103/PhysRevLett.102.206403